

KARAR AĞAÇLARI (DECISION TREES)

Veri Mad. – Hafta 11



KARAR AĞAÇLARI (DECISION TREES) VE TITANIC VERİ SETİ

Bu hafta, yorumlanabilirlik özelliği yüksek olan 'beyaz kutu' modellerden Karar Ağaçları'nı inceleyeceğiz.

Algoritmanın teorik temellerini ve bölme kriterlerini (Gini/Entropi) kavrayacağız.

Klasik bir ikili sınıflandırma problemi olan Titanic veri seti üzerinde uygulama yapacağız.

Veri ön işleme ve aşırı öğrenmeyi engelleme stratejilerini (Budama) öğreneceğiz.



BEYAZ KUTU MODELLER: KARAR AĞAÇLARI

Daha önce gördüğümüz modeller (Lojistik Regresyon, K-NN, SVM) genellikle 'kara kutu' (black-box) olarak nitelendirilebilir. Karar Ağaçları ise 'beyaz kutu' (white-box) bir modeldir, yani kararlarının nasıl alındığı tamamen şeffaf ve izlenebilirdir. Bu şeffaflık, sonuçların güvenilirliğini ve insanlar tarafından anlaşılabilirliğini artırır.



KARAR AĞAÇLARININ TEMEL MANTIĞI

Karar Ağaçları, temelde insan karar verme sürecini taklit eder. Model, ardışık 'Eğer (If)' ve 'Değilse (Else)' mantığına dayalı sorular silsilesi oluşturur. Bu sezgisel yapı, modelin çıktısını teknik olmayan kişiler için bile yorumlanabilir kılar.



Karar Ağacının temel çalışma prensibi, veri setini özelliklere (feature) dayalı olarak parçalara ayırmaktır.

Bu süreç, her adımda veri setini giderek daha küçük ve daha saf alt gruplara böler.

Bölme işlemi, özyinelemeli (recursive) bir şekilde, alt gruplar yeterince saf hale gelene kadar devam eder.



TEMEL AMAÇ: HOMOJENLİK VE SAFLIK

Bir Karar Ağacı oluşturma'nın nihai amacı, veri setini mümkün olan en saf (pure) alt gruplara ayırmaktır.

Saf bir alt grup, yaprak düğümündeki örneklerin neredeyse tamamının tek bir sınıfa ait olduğu anlamına gelir (örneğin, o yapraktaki tüm yolcuların hayatta kalması).

Bu saf alt gruplara 'yapraklar' (leaf nodes) denir ve bunlar nihai kararı temsil eder.



Kök Düğüm (Root Node): Ağacın en üstündeki düğümdür ve tüm veri setini temsil eder.

İç Düğümler (Internal Nodes): Kök düğümünden sonra gelen ve veri üzerinde bir bölme sorusunun (splitting question) sorulduğu tüm düğümlerdir.

Bu düğümler, hangi özelliğin ve hangi eşik değerinin kullanılacağına karar verilen yerlerdir.



Dallar (Branches): İç düğümlerde sorulan sorunun olası cevaplarını (Evet/Hayır) temsil eden bağlantı yollarıdır.

Yaprak Düğümler (Leaf Nodes) veya Terminal Düğümler: Ağacın son karara ulaştığı, daha fazla bölme yapılmayan uç düğümlerdir.

Bu yapraklar, alt grubun sınıf etiketini (örneğin, 'Hayatta Kaldı') atar.



BÖLME KRİTERİ (SPLITTING CRITERIA) SORUNU

Bir düğümdeyken, algoritma mevcut veriyi en iyi şekilde iki alt gruba ayıracak soruyu bulmalıdır.

'En iyi soru', bölme sonucunda oluşacak çocuk düğümlerin (child nodes) safsızlığını (impurity) en çok azaltan sorudur.

Algoritma, tüm olası özellikler ve eşik değerleri üzerinde yinelemeli olarak testler yapar.



SAFSIZLIK ÖLÇÜTLERİNE GİRİŞ

Safsızlık (Impurity), bir düğümdeki örneklerin karışıklık veya belirsizlik derecesini ölçen bir metriktir.

Eğer bir düğümdeki örneklerin yarısı bir sınıfa, yarısı diğer sınıfa aitse, safsızlık maksimumdur (0.5).

Eğer tüm örnekler aynı sınıfa aitse, safsızlık sıfırdır (mükemmel saflık). Karar ağacı algoritmaları bu safsızlığı azaltmayı hedefler.

GINİ SAFSIZLIĞI (GINI IMPURITY) - I

Gini Safsızlığı, en yaygın kullanılan algoritmaların başında gelen CART (Classification and Regression Trees) tarafından tercih edilir.

Tanım olarak, bir düğümden rastgele seçilen bir örneğin, düğümdeki sınıf dağılımına göre yanlış sınıflandırılma olasılığını ölçer.

Formülü $G = 1 - \sum_{i=1}^C p_i^2$ şeklindedir, burada p_i o sınıfa ait olma olasılığıdır.

GİNİ SAFSIZLIĞI (GINI IMPURITY) - II

Gini değeri 0 (sıfır), mükemmel saflığı ve homojenliği ifade eder. Bir düğümdeki tüm örnekler aynı sınıfa aittir (örneğin, tümü Hayatta Kaldı). İkili sınıflandırmada, Gini'nin maksimum değeri 0.5'tir (sınıfların eşit dağılımı, %50 - %50). Bölme işlemi, her zaman Gini Safsızlığını en çok azaltan özelliği seçerek yapılır.

ENTROPİ (ENTROPY) – I

Entropi, ID3 ve C4.5 gibi algoritmalar tarafından kullanılan alternatif bir safsızlık metriğidir.

Temel olarak, düğümdeki belirsizlik veya 'düzensizlik' seviyesini ölçer (Bilgi Teorisi'nden gelir).

Yüksek entropi, yüksek belirsizlik ve karışıklık anlamına gelir.

ENTROPİ (ENTROPY) – II

Entropi formülü $E = - \sum_{i=1}^C p_i \log_2(p_i)$ olarak ifade edilir. Sınıflar eşit dağıldığında (maksimum karışıklık) entropi maksimuma ulaşır (ikili sınıflandırmada 1.0). Eğer düğüm mükemmel safsa (tüm örnekler aynı sınıftan), entropi 0 olur. Bölme işlemi, entropiyi azaltmayı hedefler.

GINI VE ENTROPİ KARŞILAŞTIRMASI

Her iki kriter de benzer sonuçlar üretme eğilimindedir, ancak matematiksel olarak farklıdırlar.

Gini hesaplaması logaritma içermediği için genellikle biraz daha hızlıdır. Entropi, bazı durumlarda daha dengeli ağaçlar oluşturma eğilimindedir, ancak uygulamada aralarındaki performans farkı genellikle azdır. Scikit-learn (CART) varsayılan olarak Gini'yi kullanır, ancak 'entropy' parametresiyle değiştirilebilir.



AŞIRI ÖĞRENME (OVERFITTING) TEHLİKESİ

Karar Ağaçları, eğitim verisini mükemmel bir şekilde 'ezberleme' eğilimindedir. Eğer ağacın büyümesi sınırlandırılmazsa, her bir yaprak %100 saf olana kadar (hatta tek bir örnek içerecek kadar) dallanmaya devam edebilir. Bu durum, eğitim verisinde mükemmel performans (düşük hata) sahip, ancak test verisinde genelleme yapamayan (yüksek hata) çok derin ve karmaşık ağaçlara yol açar.

AŞIRI ÖĞRENMEDEN KAÇINMA: BUDAMA (PRUNING)

Budama (Pruning), aşırı öğrenmeyi kontrol altına almak ve daha genellenebilir (basit) bir ağaç elde etmek için uygulanan yöntemdir. İki temel yaklaşım vardır: Ön Budama (Pre-pruning) ve Son Budama (Post-pruning).

ÖN BUDAMA (PRE-PRUNING) HİPERPARAMETRELERİ

Ön budama, ağaç daha oluşurken belli kurallar koyarak büyümesini durdurur. En yaygın kullanılan kısıtlamalar hiperparametreler aracılığıyla uygulanır:

- max_depth: Ağacın kökten yaprağa maksimum derinliğini belirler.
- min_samples_leaf: Bir düğümün yaprak olabilmesi için sahip olması gereken minimum örnek sayısını belirler.

max_depth PARAMETRESİNİN ÖNEMİ

max_depth değeri ne kadar küçükse, ağaç o kadar sığ (basit) olur ve o kadar az detay öğrenir.

Çok küçük bir değer (örneğin, 2 veya 3) yetersiz öğrenmeye (underfitting) yol açabilir.

Titanic gibi veri setlerinde, max_depth=5 gibi orta düzey bir sınırlandırma, genellikle iyi bir denge sağlar.

min_samples_leaf PARAMETRESİ

Bu hiperparametre, bir düğümün bölme işleminden sonra oluşacak yapraklarda bulunması gereken minimum veri noktasını garanti eder. Eğer bir bölme, kısıtlamanın altında veri noktasına sahip bir yaprak oluşturuyorsa, o bölme yapılmaz. Bu, aşırı özelleşmiş (tekil) kararların oluşmasını engeller.

HAFTANIN VERİ SETİ: RMS TITANIC

Veri seti, 1912 yılında batan RMS Titanic gemisindeki yolcuların seyahat ve demografik bilgilerini içerir.
Temel hedef: İkili Sınıflandırma. Bir yolcunun hayatta kalıp kalmadığını tahmin etmektir (Hayatta Kaldı: 1, Kalmadı: 0).
Eğitim seti genellikle yaklaşık 891 yolcu kaydını içerir.

Karar Ağaçları'nın 'beyaz kutu' yapısı, Titanic verisinde tarihi bir bağlam sunar. Modelin ürettiği kurallar ('Eğer Cinsiyet == Kadın İSE...'), facianın dinamikleri hakkında doğrudan içgörüler sağlar. Bu, modelin neden bu kararı verdiğini anlamak açısından K-NN veya SVM'den üstündür.

Karar Ağacının kök düğümde veya ilk birkaç adımda cinsiyet özelliğini kullanması beklenir.

Bu, modelin veri içinde yatan en güçlü hayatta kalma sinyalini (kurtarma protokolleri) yakaladığını teyit eder.

Bu gözlem, makine öğrenimi modelinin tarihsel olaylarla uyumlu sonuçlar ürettiğini gösterir.

Veri seti, veri tiplerinin çeşitliliğini göstermek için mükemmeldir:
Sayısal (Numerical): 'Age' (Yaş) ve 'Fare' (Bilet Ücreti).
Kategorik (Categorical): 'Sex' (Cinsiyet) ve 'Embarked' (Biniş Limanı).
Sıralı (Ordinal): 'Pclass' (Bilet Sınıfı – 1., 2., 3. Sınıf).

Veri seti, gerçek dünya problemlerinde sıkça karşılaşılan eksik veri (NaN) sorununu içerir.

Özellikle 'Age' (Yaş) sütununda önemli miktarda eksik gözlem bulunur. Bu durum, veri ön işleme aşamasında 'Eksik Veri Doldurma' (Imputation) pratiği yapmayı zorunlu kılar.



UYGULAMA BAŞLANGICI: PYTHON VE SCIKIT-LEARN

Karar Ağacı uygulamaları genellikle Python ortamında, Google Colab veya Jupyter Notebook kullanılarak yapılır.
Veri yükleme, pandas kütüphanesindeki `read_csv()` komutu ile gerçekleştirilir.
Bu aşamayı kritik bir veri ön işleme (Preprocessing) aşaması takip eder.

VERİ ÖN İŞLEME (PREPROCESSING) MİMARİSİ

Karar Ağacı (ve çoğu makine öğrenimi modeli), metin tabanlı kategorik verileri (örneğin 'Male', 'Female') doğrudan işleyemez.

Bu verilerin modelin anlayabileceği sayısal formata dönüştürülmesi gerekmektedir.

Eksik değerler (NaN) de modelin çökmemesi için doldurulmalıdır.

KATEGORİK VERİ DÖNÜŞÜMÜ - I

One-Hot Encoding, nominal (sıralı olmayan) kategorik verileri sayısal hale getirmenin en yaygın yoludur. Bu teknikte, her benzersiz kategori için yeni bir ikili (binary) sütun oluşturulur. Örneğin, 'Sex' sütunu 'Sex_male' ve 'Sex_female' olarak ikiye ayrılır (0 veya 1).

KATEGORİK VERİ DÖNÜŞÜMÜ - II

Ordinal Encoding, kategoriler arasında doğal bir sıra (rütbe) olduğu zaman kullanılır.

Örneğin, Pclass (1. > 2. > 3. Sınıf).

scikit-learn'deki OrdinalEncoder, kategorileri doğrudan sayısal değerlere (1, 2, 3) eşler.

Karar Ağaçları, sıralı verilerle bu sayede mantıksal bölmeler yapabilir.

EKSİK VERİ DOLDURMA (IMPUTATION) STRATEJİLERİ - I

'Age' sütunundaki eksik (NaN) değerleri gidermek için doldurma (imputation) gerekir.

En basit stratejiler, tüm veri setindeki yaş ortalamasını (mean) veya medyanını (median) kullanarak boşlukları doldurmaktır.

Medyan, aykırı değerlerden (outliers) daha az etkilendiği için genellikle tercih edilir.



EKSİK VERİ DOLDURMA (IMPUTATION) STRATEJİLERİ - II

Kategorik sütunlarda (örneğin 'Embarked') eksik veri varsa, genellikle en sık görülen değer (mode) ile doldurma yapılır. Alternatif olarak, eksik değerler için 'Bilinmiyor' (Unknown) gibi yeni bir kategori oluşturulabilir. Bu, modelin eksik olma durumunu da bir bilgi olarak kullanmasına olanak tanır.



MODEL KURULUMU VE İNŞASI

Scikit-learn kütüphanesinde Karar Ağaçları, `sklearn.tree` içindeki `DecisionTreeClassifier` çağrılarak oluşturulur.

Model kurulumunda bölme kriteri (`criterion`) seçilebilir:
`criterion='gini'` (varsayılan) veya `criterion='entropy'`.

Aşırı öğrenmeyi kontrol etmek için ön budama hiperparametreleri burada tanımlanır (örneğin `max_depth`).

MODELİN EĞİTİMİ (FITTING)

Model, `fit()` komutu kullanılarak önceden işlenmiş eğitim verileri (`X_train`) ve hedef değişken (`y_train`) ile eğitilir.

Bu süreçte algoritma, en yüksek Bilgi Kazancını (veya Gini azalmasını) sağlayacak ardışık bölmeleri oluşturur.

Eğitimden sonra ağaç yapısı sabitleşir ve yeni veriler üzerinde tahmin yapmaya hazırdır.

HİPERPARAMETRE AYARI: max_depth=5

Titanic gibi küçük bir veri setinde, ağacın sonsuz derinleşmesini engellemek kritik öneme sahiptir.

max_depth=5 gibi bir sınır koymak, modelin sadece en güçlü 5 hiyerarşik kuralı öğrenmesini sağlar.

Bu, test verisi üzerinde daha robust (sağlam) ve genellenebilir bir performans sağlar.



MODEL ÇIKTILARININ GÖRSELLEŐTİRİLMESİ

Karar Ağaçları'nın benzersiz avantajı, öğrendiđi tüm 'Eđer-Değilse' kurallarının görsel bir ağaç diyagramı olarak incelenebilmesidir. Bu görsel, kökten yaprağı kadar her bir kararı ve bu kararın hangi özelliklere dayandığını açıkça gösterir.

GÖRSELLEŞTİRME YÖNTEMİ I: `plot_tree()`

`sklearn.tree.plot_tree()` komutu, Karar Ağacını hızlıca bir Matplotlib grafiği olarak çizer.

Okunabilirliği artırmak için 'feature_names' (kullanılan özelliklerin isimleri) ve 'class_names' (sınıf etiketleri, örneğin 'Survived', 'Not Survived') parametreleri kullanılır.



GÖRSELLEŞTİRME YÖNTEMİ II: `export_graphviz()` - I

Daha yüksek kaliteli, etkileşimli ve detaylı görselleştirme için GraphViz kütüphanesi kullanılır.

Model, `sklearn.tree.export_graphviz()` komutu ile GraphViz formatında (`.dot` uzantılı) bir metin dosyasına dışa aktarılır.



GÖRSELLEŞTİRME YÖNTEMİ II: `export_graphviz()` - II

Dışa aktarılan .dot dosyası, GraphViz motoru kullanılarak görselleştirilir. Bu çıktılar, her düğümdeki Gini/Entropi değerlerini, örnek sayısını (samples) ve sınıf dağılımını gösterir. 'filled=True' parametresi, düğümleri baskın sınıfa göre renklendirerek anlaşılabilirliği artırır.



KÖK DÜĞÜM ANALİZİ

Oluşan ağacın kök düğümü (en üstteki soru), veri setini en iyi ayıran özelliği gösterir.

Titanic veri setinde, bu neredeyse her zaman 'Cinsiyet' (Sex) özelliğidir. Bu ilk split, hayatta kalma olasılığını belirleyen en güçlü hiyerarşik faktörü ortaya koyar.



Karar Ağaçları, sadece hata matrisi ve sınıflandırma raporu sağlamakla kalmaz.

Model, karar verme sürecine hangi özelliklerin kantitatif olarak daha fazla katkıda bulunduğunu ölçmemizi sağlar.

Bu, modelin tahminlerine güveni artıran ek bir yorumlanabilirlik katmanıdır.



ÖZELLİK ÖNEMİ: TEORİK TEMEL

Bir özelliğin önemi, o özelliğin kullanıldığı tüm düğümlerde toplam safsızlığı (Gini veya Entropi) ne kadar azalttığına bağlı olarak hesaplanır. Özelliğin toplam safsızlık azalmasına katkısı ne kadar yüksekse, o özellik model için o kadar önemlidir. Bu değerler, modelin 'feature_importances_' özniteliği aracılığıyla erişilebilir.

feature_importances_ ÖZNİTELİĞİ

Eğitilmiş Karar Ağacı modeli, her bir giriş özelliği için 0 ile 1 arasında bir önem puanı dizisi (array) döndürür.

Bu puanların toplamı daima 1'e eşittir.

Özellik adları (örneğin 'Age', 'Fare', 'Sex_male') bu puanlarla eşleştirilerek yorumlanır.



ÖZELLİK ÖNEMİNİN GÖRSELLEŞTİRİLMESİ

Elde edilen önem değerleri, özellik adlarıyla birlikte bir Matplotlib çubuk grafiği (`plt.bar()`) kullanılarak görselleştirilir.

Bu grafik, en önemli özelliklerin en uzun çubuklara sahip olmasını sağlayarak modelin odak noktalarını hızla belirlemeye yardımcı olur.

TITANIC ÖRNEĞİNDE ÖNEMLİ ÖZELLİKLER

Titanic veri setinde tipik olarak en önemli çıkan özellikler şunlardır:

'Sex' (Cinsiyet): Kurtarma protokollerinin doğası gereği en güçlü sinyaldir.

'Pclass' (Bilet Sınıfı): Üst sınıflara öncelik verilmesi nedeniyle yüksek önem taşır.

'Fare' veya 'Age' gibi diğer özellikler de önemlidir, ancak genellikle ilk iki kadar dominant değildir.



SONUÇ VE TARİHSEL UYUMLULUK

Özellik önem grafiğinin 'Cinsiyet' ve 'Bilet Sınıfı'nı en önemli olarak göstermesi, modelin veri içindeki en güçlü sinyalleri yakaladığını kanıtlar.

Bu bulgular, facianın tarihsel gerçekleriyle (kadınlara ve üst sınıf yolculara öncelik) tam olarak uyumludur.

Bu durum, Karar Ağacının hem güçlü bir sınıflandırıcı hem de değerli bir içgörü aracı olduğunu gösterir.



KARAR AĞAÇLARI: AVANTAJLAR VE DEZAVANTAJLAR

Avantajlar:

Kolay yorumlanabilirlik ('beyaz kutu').

Veri normalizasyonu veya ölçeklendirmesi gerektirmez.

Kategorik ve sayısal verileri aynı anda işleyebilir.

Dezavantajlar:

Aşırı öğrenmeye eğilimlidir (budama gerektirir).

Küçük veri değişikliklerine karşı kararsız olabilir (yüksek varyans).

Optimal ağaç bulmak NP-hard bir problemdir (pratik algoritmalar genellikle ağgözlüdür).

EK TEKNİKLER: ENSEMBLE METOTLARINA GİRİŞ

Karar Ağaçlarının kararsızlık (yüksek varyans) dezavantajını gidermek için Topluluk Öğrenimi (Ensemble Learning) teknikleri geliştirilmiştir. Bu teknikler, birden fazla Karar Ağacını birleştirerek daha sağlam ve genellenebilir modeller oluşturur. Önemli Örnekler: Rastgele Orman (Random Forest) ve Gradyan Arttırma (Gradient Boosting).

RASTGELE ORMAN (RANDOM FOREST)

Rastgele Orman, birçok bağımsız Karar Ağacının aynı veri seti üzerinde eğitildiği bir topluluk yöntemidir.

Her ağaç hafifçe farklı veri alt kümeleriyle (Bagging) ve farklı özellik alt kümeleriyle eğitilir (Feature Randomness).

Nihai karar, tüm ağaçların verdiği kararların ortalaması veya çoğunluğu alınarak belirlenir.

Bu, varyansı azaltarak aşırı öğrenmeyi büyük ölçüde engeller.



GRADYAN ARTTIRMA (GRADIENT BOOSTING)

Gradyan Arttırma (örneğin XGBoost, LightGBM), ağaçların seri halinde oluşturulduğu bir yöntemdir.

Her yeni ağaç, kendisinden önceki ağacın yaptığı hataları düzeltmeyi öğrenir. Bu teknik, genellikle Karar Ağaçları tabanlı en yüksek performanslı modelleri üretir (daha düşük sapma ve varyans sağlar).



DİKKAT EDİLMESİ GEREKEN DETAYLAR

Sayısal (sürekli) bir özellik (örneğin Yaş) kullanılarak bölme yapılırken, algoritma optimal bir eşik değeri (threshold) bulur.

Örneğin: 'Yaş ≤ 28.5 İSE' sorusu.

Bu eşik, safsızlığı en çok azaltacak noktayı belirlemek için veri üzerindeki tüm olası bölme noktaları test edilerek bulunur.

One-Hot Encoding, birçok farklı kategoriye sahip bir sütun için çok sayıda yeni sütun (boyut) oluşturabilir.

Bu durum, ağacın daha fazla bölme seçeneğine sahip olmasına neden olur, ancak hesaplama maliyetini artırır.

Titanic'te bu sorun azdır ('Embarked' 3-4 kategori), ancak büyük veri setlerinde dikkat edilmelidir.



GERÇEK ZAMANLI KARAR SİSTEMLERİ

Karar Ağaçları, basitliği ve hızı nedeniyle hızlı karar verme sistemlerinde kullanılır.

Örnekler:

Kredi risk skoru belirleme (açık kurallar gereklidir).

Tıbbi teşhis destek sistemleri.

Müşteri kaybı (churn) tahmini (hangi faktörlerin ayrılmaya yol açtığını anlamak).



TITANIC'TEKİ DENGİ PROBLEMİ

Titanic veri setinde hayatta kalan yolcu sayısı (yaklaşık %38) hayatta kalmayanlardan daha azdır.

Bu sınıf dengesizliği (Class Imbalance), modelin hayatta kalmayanları tahmin etmede daha iyi olmasına neden olabilir.

Bu tür durumlarda F1 skoru, Hassasiyet (Precision) ve Duyarlılık (Recall) gibi metrikler sadece doğruluk (Accuracy) yerine kullanılmalıdır.

KARAR AĞAÇLARINDA ÇARPITMA (BIAS)

ID3 gibi bazı eski Karar Ağacı algoritmaları, çok sayıda benzersiz değere sahip kategorik özelliklere (yüksek kardinalite) karşı doğal bir çarpıtmaya sahiptir. Bu özellikler, safsızlığı kolayca sıfıra indirme potansiyeline sahiptir, ancak bu yapay bir saflık olabilir. C4.5 gibi algoritmalar, bu çarpıtmayı düzeltmek için Gain Ratio (Kazanç Oranı) kullanır.



SINIFLANDIRMA VE REGRESYON AĞAÇLARI (CART)

CART algoritması (Classification and Regression Trees), hem sınıflandırma hem de regresyon görevleri için kullanılabilir.

Sınıflandırmada (Titanic), Gini veya Entropi kullanılır.

Regresyonda, safsızlık yerine düğümdeki varyansın (hata kareleri toplamı) azalması hedeflenir.

Regresyon ağacının yaprağı, o bölgedeki ortalama hedef değeri tahmin eder.

PRUNING TEKNIĞİ: MALİYET KARMAŞIKLIĞI BUDAMASI

Son budama, tam olarak büyümüş (aşırı öğrenmiş) bir ağaç oluşturulduktan sonra, gereksiz dalların kesilmesidir.

Maliyet Karmaşıklığı Budaması (Cost-Complexity Pruning) en yaygın yöntemdir.

Bu, model karmaşıklığı ile genelleme performansı arasında bir denge kurar. Alfa (α) parametresi, karmaşıklığın cezalandırılma derecesini kontrol eder.

KARAR SINIRLARI (DECISION BOUNDARIES)

Karar Ağaçları, özellik uzayını eksenlere paralel dikdörtgen bölgelere böler. Her bir yaprak düğümü, bu uzaydaki tek bir dikdörtgeni temsil eder ve o dikdörtgenin içindeki tüm noktalar aynı sınıf etiketiyle atanır. Aşırı derin bir ağaç, çok karmaşık, düzensiz ve küçük dikdörtgenler oluşturur.

TITANIC ÖRNEĞİ: ETKİLEŞİM ETKİSİ

Karar Ağaçları, doğal olarak özellikler arasındaki etkileşimleri yakalar. Örneğin, 'Cinsiyet = Kadın' koşulunun altında, ağaç 'Bilet Sınıfı ≤ 1.5 ' diye sorabilir.

Bu, kadın olmanın etkisinin, bilet sınıfına bağlı olarak değiştiğini (yani etkileşim olduğunu) gösterir.



GÜRÜLTÜLÜ VERİ VE KARAR AĞAÇLARI

Eğer veri seti çok gürültülüyse (noisy) veya aykırı değerler içeriyorsa, Karar Ağaçları kolayca aşırı öğrenme eğilimine girebilir.

Model, gürültüyü bile mükemmel bir şekilde sınıflandırmak için gereksiz dallar oluşturabilir.

Bu, daha fazla budama yapılmasını veya topluluk yöntemlerinin kullanılmasını gerektirir.



ÖZET: KARAR AĞAÇLARI VE TITANIC

Karar Ağaçlarının sezgisel, hiyerarşik yapısını ve beyaz kutu özelliğini öğrendik. Safsızlık ölçütleri (Gini ve Entropi) ve Bilgi Kazancı kavramlarını inceledik. Aşırı öğrenme, budama ve min_samples_leaf gibi hiperparametrelerin önemini kavradık. Titanic veri setinde veri ön işleme (One-Hot Encoding, Imputation) ve özellik önemini görselleştirme adımlarını uyguladık.

Öğrenme sürecinizi derinleştirmek için:

Random Forest ve Boosting (XGBoost, CatBoost) algoritmalarını detaylı inceleyin.

Karar Ağaçları'nın regresyon problemleri için nasıl kullanıldığına odaklanın. Son budama yöntemlerini ve maliyet karmaşıklığı budamasını uygulayın.



UYGULAMA KODU PARÇASI: IMPUTATION

Pandas kütüphanesi ile basit doldurma:

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

Scikit-learn SimpleImputer kullanımı (Pipeline için ideal):

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy='median')
```

```
X_train['Age'] = imputer.fit_transform(X_train[['Age']])
```

UYGULAMA KODU PARÇASI: ONE-HOT ENCODING

Pandas get_dummies kullanımı:

```
df_processed = pd.get_dummies(df, columns=['Sex', 'Embarked'],  
drop_first=True)
```

'drop_first=True' kullanılması, çoklu doğrusallık (multicollinearity) sorununu engeller ve ağacın yalnızca tek bir kategori üzerinden bölme yapmasını sağlar.

UYGULAMA KODU PARÇASI: MODEL EĞİTİMİ

```
from sklearn.tree import DecisionTreeClassifier
# Model oluşturma ve ön budama (max_depth)
dtc = DecisionTreeClassifier(criterion='gini', max_depth=5,
random_state=42)
# Eğitim
dtc.fit(X_train_processed, y_train)
```

```
from sklearn.metrics import classification_report  
y_pred = dtc.predict(X_test_processed)  
print(classification_report(y_test, y_pred))
```

Bu rapor, modelin her bir sınıf (Hayatta Kaldı/Kalmadı) için precision, recall ve F1 skorlarını sunar.

Precision (Hassasiyet): Modelin 'Hayatta Kaldı' dediği kişilerden gerçekten hayatta kalanların oranı.

Recall (Duyarlılık): Gerçekten hayatta kalanların, model tarafından doğru tahmin edilen oranı.

F1 Skoru: Precision ve Recall'ın harmonik ortalamasıdır; dengesiz veri setlerinde daha güvenilir bir metrik sunar.



KARAR AĞAÇLARI (DECISION TREES)

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

